40. Observability: Logs, Metrics, Tracing

40. Observability: Logs, Metrics, Tracing

Phase: Phase 6 — Systems and infrastructure
Estimated time: 1–2 days
Slug: `40-observability`

Learning objectives

- Emit structured logs
- Define RED/USE metrics
- Propagate trace IDs
- Make dashboards actionable

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Logs, metrics, and traces explain production. Structured observability turns incidents from guesswork into evidence.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Structured logs: key=value or JSON fields.
- Metrics: counters, histograms, gauges.
- Traces: spans across services with context propagation.
- Correlate with request IDs.
- Cardinality kills metrics systems — label carefully.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-observabilit && cd /tmp/gw-observabilit
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import (
    "log/slog"
    "os"
)

func main() {
    logger := slog.New(slog.NewJSONHandler(os.Stdout, nil))
    logger.Info("request done", "route", "/health", "ms", 3)
}
```

Run it:

```
go run .
```

Expected output:

```
{"time":"...","level":"INFO","msg":"request done","route":"/health","ms":3}
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Log with slog JSON handler.
2. Increment a conceptual counter around a function.
3. Pass a request ID through context (sketch).

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Three pillars

- **Logs**: discrete events (structured JSON)
- **Metrics**: aggregates (counters, histograms)
- **Traces**: request journeys across services

```
slog.Info("request completed", "method", r.Method, "path", r.URL.Path, "status",
status, "dur_ms", ms)
```

Metrics that matter

For services: rate, errors, duration (RED). Histograms for latency percentiles.

Tracing

Pass trace context in headers; OpenTelemetry is the common standard. Sample wisely in production.

Failure modes to watch while coding

- Println debugging as the only strategy

- High-cardinality metric labels (user IDs)
- Logging secrets/PII

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- logging/structlog ≈ slog.
- Prometheus client patterns similar.
- Print debugging does not scale — structure early.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

ALERT on symptoms (error rate, latency) not only causes.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Observability: Logs, Metrics, Tracing" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Println debugging as the only strategy
- High-cardinality metric labels (user IDs)
- Logging secrets/PII

Exercises

Work in `exercises/40-observability/`. Read that folder's `README.md` before you code.

What you will build

Implement logging helper or metric wrapper the tests expect.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/40-observability/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. RED metrics are?
2. Why structured logs?
3. Danger of high cardinality?

Further reading

- https://pkg.go.dev/log/slog
- https://opentelemetry.io/